Accuknox Assignment

Submitted By : Subash S
Assignment Submission: Problem Statement 1 & Problem Statement 2
Email: subashs0027@gmail.com | **Phone**: *90425 96749* | **Location**: Chennai, India
GitHub: *https://github.com/subashs27* | **Resume**: RESUME(S_SUBASH).pdf

Problem Statement 1
Q1 : API Data Retrieval and Storage: You are tasked with fetching data from an external REST API, storing it in a local SQLite database, and displaying the retrieved data. The API provides a list of books in JSON format with attributes like title, author, and publication year.
ANS : Subash-AccuKnox-Q1-ANS.ipynb

Q2: Data Processing and Visualization: Given a dataset containing information about students' test scores, fetch the data from an API, calculate the average score, and create a bar chart to visualize the data.
ANS : subash-AccuKnox-ANS-Q2.ipynb

Q3: CSV Data Import to a Database: Write a Python script that reads data from a CSV file containing user information (e.g., name, email) and inserts it into a SQLite database.
ANS : Subash-AccuKnox-Q3-ANS.ipynb

Send a link to the most complex Python code you have written
ANS :
https://github.com/subashs27/AI-Interview-Management-System/blob/main/interview/interview_manager.py

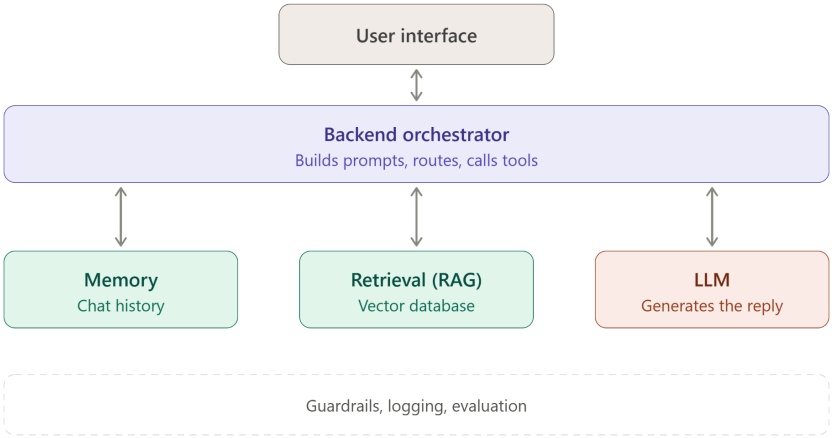
Send a link to the most complex database code you have written
ANS :
https://github.com/subashs27/AI-Interview-Management-System/blob/main/database/database.py

Problem statement- assignment 2
Q1 : Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]

ANS : B = can code under supervision

Q2 : What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level

ANS :



Simple flow:
The user asks a question → backend loads chat history → searches the vector database → builds the prompt → LLM generates the answer → safety check → answer shown to the user.

Architecture Explanation :

1. **user interface**, which is the chat window where the user types a question. This can be a website, a mobile app, or even a WhatsApp or Slack bot.
2. The message then goes to the **backend**. This is a normal server, usually written in Python with something like FastAPI or Flask. It acts as the coordinator. It receives the question, gathers everything the model needs, sends it to the LLM, and returns the answer to the user. Libraries like LangChain or LlamaIndex are often used here to make this easier.
3. The LLM does not remember earlier messages by itself, so we need **memory**. The backend stores the recent conversation, in a database like Redis or SQL, and sends it along with each new question. That is how the bot understands a follow-up like “what about the second one?”.
4. The LLM also does not know our own data, such as company documents or college circulars. For this we use **retrieval**, which is also called RAG (Retrieval-Augmented Generation). We break our documents into small chunks, convert them into embeddings (numbers that capture meaning), and store them in a **vector database**. When a user asks something, we search for the chunks closest in meaning to the question and add them to the prompt. The model then answers from real information instead of guessing, which reduces wrong answers.
5. All of this is put together in a **prompt**: instructions for the bot (its role and tone), the chat history, the retrieved text, and the user’s question. This prompt goes to the **LLM**, which writes the final reply.

6. Finally, a production chatbot needs **guardrails and monitoring**. Guardrails check that the answer is safe and on topic. Monitoring logs the conversations so we can track cost, speed and answer quality, and improve the bot over time.

Example: For a college helpdesk bot, a student asks “What is the last date to pay fees?”. The backend finds the matching paragraph from the fee circular, sends it to the LLM along with the question, and the LLM replies with the date back to the user .

Q3: Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why? Note: These are subjective questions and could be anywhere from half a page to multiple pages.

ANS :

Vector databases

A normal database finds data by exact match. If I search for “fee payment last date”, it only returns rows that contain those exact words. A vector database finds data by meaning.

It works like this. An embedding model converts a piece of text (or an image, or audio) into a long list of numbers called a vector. Texts with similar meaning end up with similar vectors. “Last date to pay fees” and “deadline for tuition payment” use different words, but their vectors are very close. A vector database stores millions of these vectors and, given a new question, quickly finds the closest ones. This is called **similarity search**, and it usually uses measures like cosine similarity.

Comparing a query against every stored vector would be too slow at large scale, so these databases use special indexes such as **HNSW**, which find very close matches (not always the exact closest) in milliseconds. Most of them also store extra information with each vector, like the source document, department or date, so you can filter results (for example, “only circulars from 2026”).

They are mainly used in RAG chatbots, semantic search, recommendation systems and duplicate detection. In a chatbot, the vector database is what lets the LLM see the right documents before it answers.

Simple flow :
Storing: Documents → split into chunks → create embeddings → save in vector database.

Searching: User question → create embedding → similarity search → top matching chunks → sent to the LLM.

Q4 - Hypothetical Problem :

If I had to select a vector database ,

I'll use a problem from my own project. In my Enterprise AI Knowledge Assistant, I used FAISS to search across 12 policy documents. It worked well at that size. Now suppose a company wants to roll the same assistant out to the whole organisation, with around 5,000 documents from HR, Finance, IT and Legal, and with updates every week. I would choose **Qdrant** for this.

Why Qdrant

- **Metadata filtering.** In a company, an employee should only see documents meant for their department. Qdrant lets me attach details like department, document type and access level to every vector and filter on them during the search. FAISS only stores vectors, so I would have to build this filtering myself.
- **Hybrid search.** My current project already combines vector search with BM25 keyword search. Qdrant supports both dense and keyword-style (sparse) vectors, so I can keep this approach inside one database, and still use the Cross-Encoder to re-rank the top results.
- **Easy updates.** Policies change often. In Qdrant I can add, update or delete a document's chunks directly. With FAISS, updating and deleting is awkward and often means rebuilding the index.
- **Cost and data privacy.** It is open source and I can self-host it with Docker. Company policy documents stay inside the company, and there is no per-month fee for the database.

Why not the others

- **FAISS** is great for a small project like my current one, but it is only a search library, with no built-in storage or filtering.
- **ChromaDB** is very easy for prototypes, but I would want stronger filtering and production features at this scale.
- **Pinecone** is fully managed and convenient, but it is paid and the data sits with a third party, which is a concern for internal company documents.

So for a small prototype I would stay with FAISS or ChromaDB, and when it grows into a real company-wide system with access rules, I would move to Qdrant.

THANK YOU